



Data Infrastructure for Machine Learning

(v1.00)

Week 6: October 9

Jimmy Lin
David R. Cheriton School of Computer Science
University of Waterloo

These slides are available at <http://lintool.github.io/bigdata-2025f/>

This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International
See <https://creativecommons.org/licenses/by-nc-sa/4.0/> for details



Key Questions

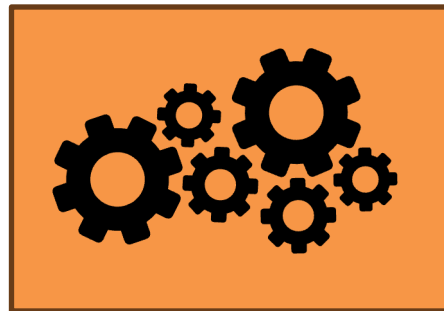
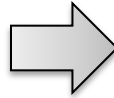
What are the key components of an ML solution?

How is the supervised machine learning problem formulated?

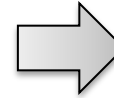
What roles do data platforms and data engineering play?

Instance

amazing spot for good
food & a fun time 🍕🍜
they offer a super unique
dine-in experience with
their interactive tables!
also love that they have
innovative weekly feature
dishes 😊



Model

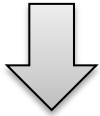


Prediction



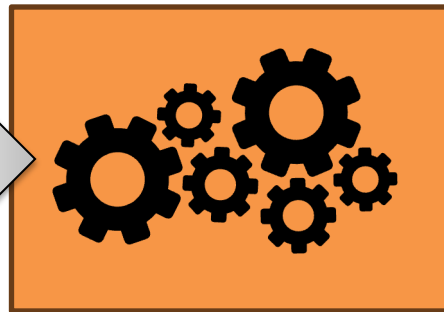
Instance

amazing spot for good
food & a fun time 🍕🍷
they offer a super unique
dine-in experience with
their interactive tables!
also love that they have
innovative weekly feature
dishes 😊



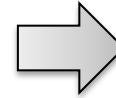
$$\mathbf{x}_i = [x_1, x_2, x_3, \dots, x_d]$$

Feature Vector



Model

$$f : X \rightarrow Y$$



Prediction

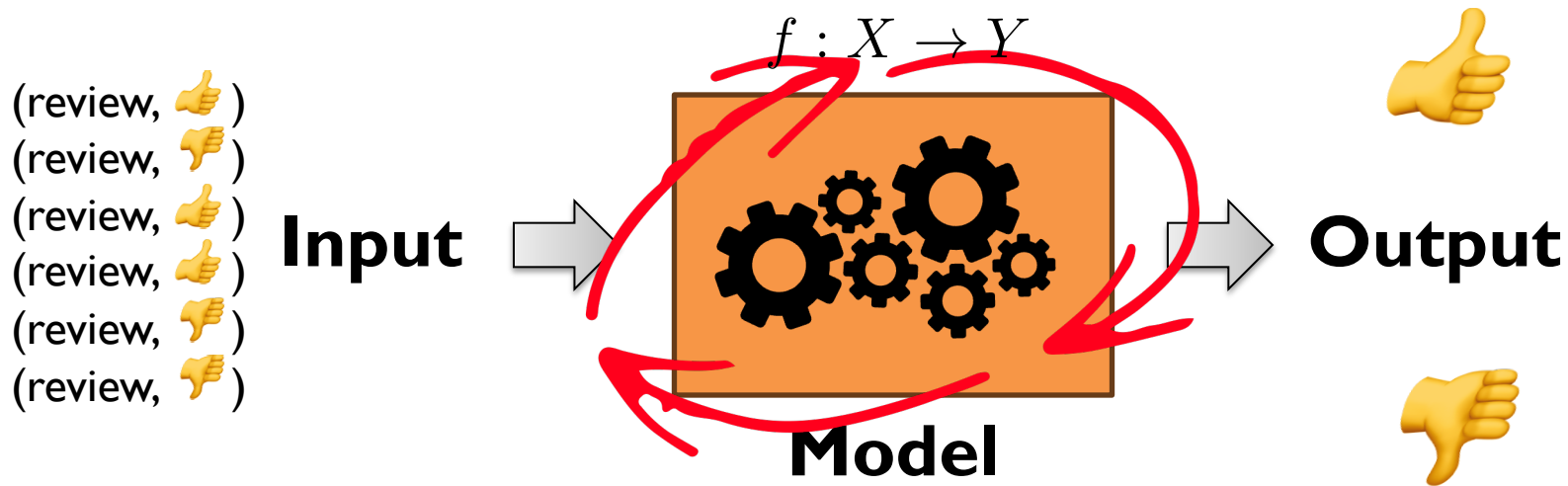


$$y \in \{0, 1\}$$

Components of an ML solution

(data, features, model, optimization)

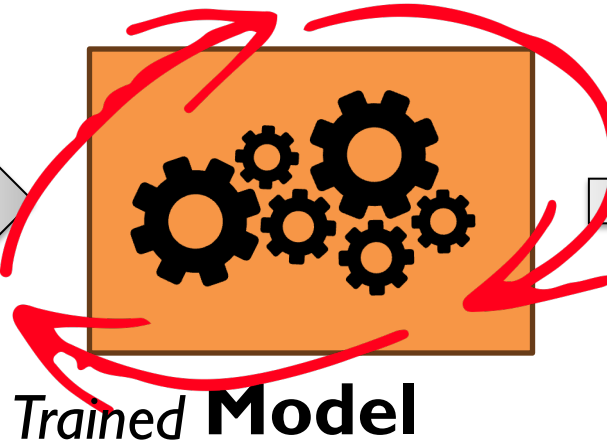
Model *learns* from the data



Machine learning algorithm
adjusts the model *parameters*

(review, 👍)
(review, 👎)
(review, 👍)
(review, 👍)
(review, 👎)
(review, 👎)

Input



Output

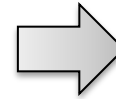
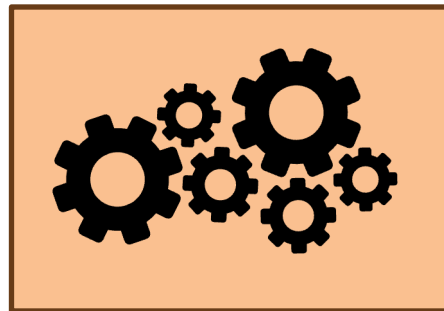
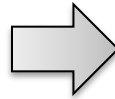


Deployment



Inference / Prediction

A group of us stopped by
yesterday afternoon to
enjoy an outdoor lunch.
The food was da bomb.



Trained **Model**

Got it!

Gather training data

Train model

Deploy model

Goal for today: Dispel the myth of what a
data scientist *actually* does.

Applied {ML, AL} Researcher, etc.

Origins:

Applied ML in Academia

Download interesting dataset (comes with the problem)

Run baseline model

Train/Test

Build better model

Train/Test

Does new model beat baseline?

Yes: publish a paper!

No: try again!

Cool, you do that in industry,
except you get paid a lot more!

Harvard
Business
Review

DATA

Data Scientist: The Sexiest Job of the 21st Century

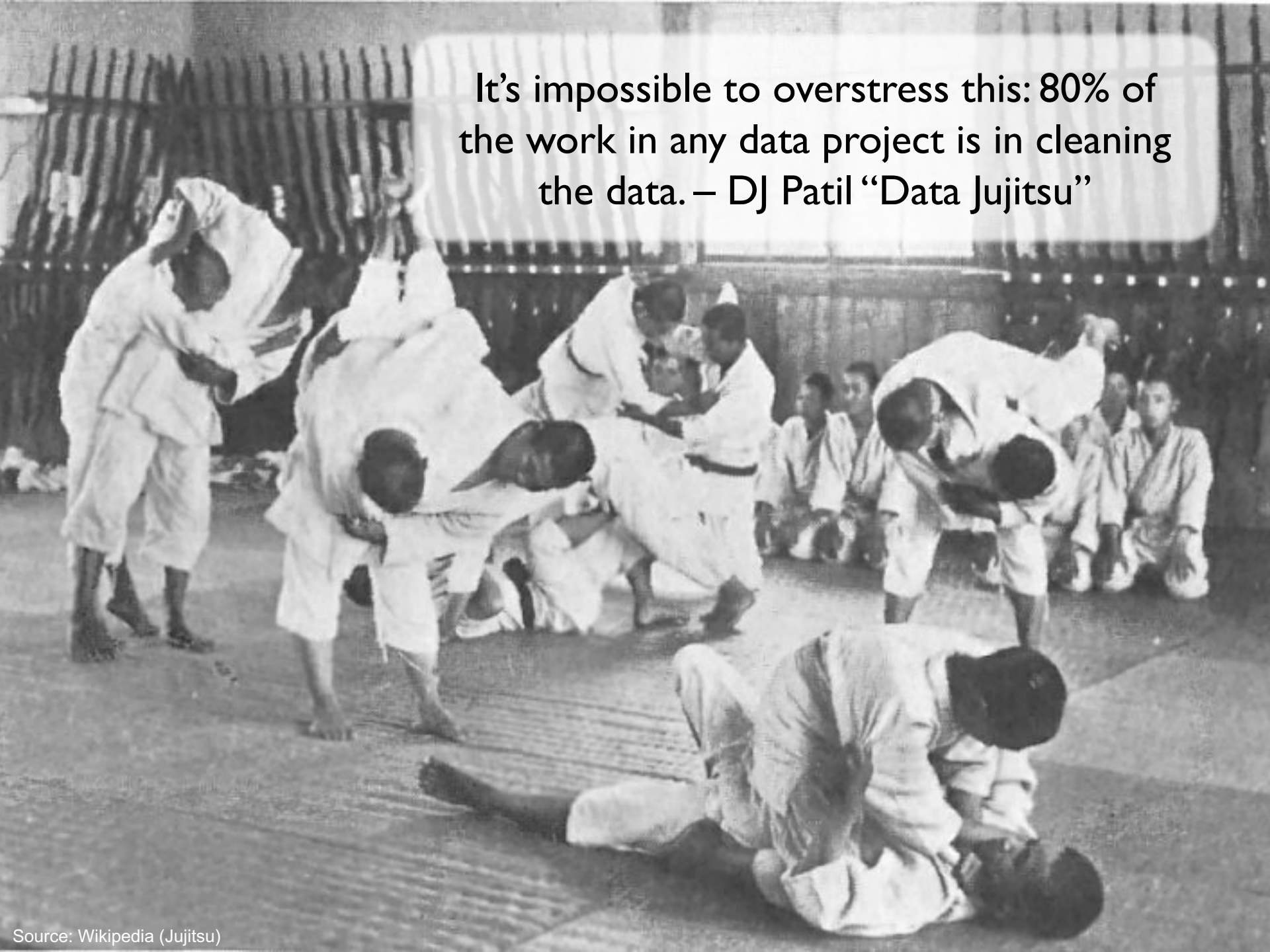
by Thomas H. Davenport and D.J. Patil

FROM THE OCTOBER 2012 ISSUE

Goal for today: Dispel the myth of what a
data scientist *actually* does.
Applied {ML, AL} Researcher, etc.

<pause/>

It's impossible to overstress this: 80% of the work in any data project is in cleaning the data. – DJ Patil “Data Jujitsu”





TECHNOLOGY

For 'Big Data' Scientists, Hurdle to Insights Is 'Janitor Work'

By STEVE LOHR AUG. 17, 2014



Monica Rogati, Jawbone's vice president for data science, with Brian Wilt, a senior data scientist.
Peter DaSilva for The New York Times

Raw data
(e.g., from ELT)



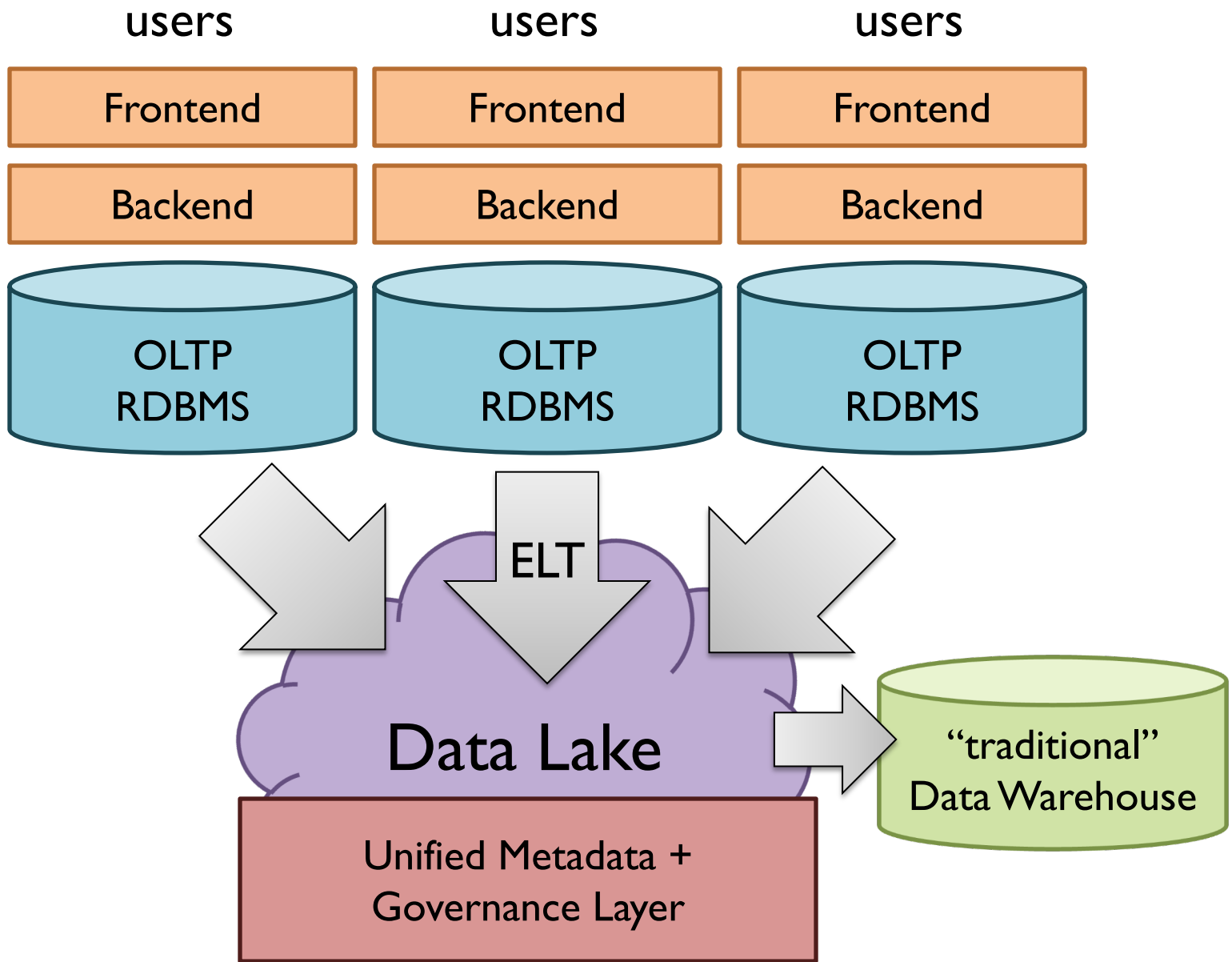
High-quality training data

GI/GO

Gather training data

Train model

Deploy model



Lakehouse

Where's the data?

Who owns that data?

Can I have access to it?

What does this field mean?

Why does it have this value?

What's all this \$#@%&?

Wait, this isn't *actually* what I need.



On finding things...



P. Oscar Boykin
@posco



Following

OH: "... so to recap, tweets are statuses,
favorites are favourings, retweets are
shares."

↩ Reply ↻ Retweet ★ Favorite ... More

On naming things...

CamelCase

smallCamelCase

snake_case

camel_Snake

dunder__snake

uid *UserId*

userId

userid

user_id

user_Id



Bill Graham
@billgraham

Yesterday I had a run in with the camel_Snake in our code. Today, I came across the feared dunder__snake. Yow! /via @THISWILLWORK

Reply Retweet Favorite More

1
FAVORITE



10:46 PM - Sep 12, 2012 from SoMa, San Francisco

Raw data
(e.g., from ELT)



High-quality training data

Gather training data

Train model

Deploy model

Assume you've got a high-quality labeled dataset...

Gather training data

Train model

Deploy model

The Task

Given: $D = \{(\mathbf{x}_i, y_i)\}_i^n$



label

feature vector

$$\mathbf{x}_i = [x_1, x_2, x_3, \dots, x_d]$$

$$y \in \{0, 1\}$$

Induce: $f : X \rightarrow Y$

Such that loss is minimized

$$\frac{1}{n} \sum_{i=0}^n \ell(f(\mathbf{x}_i), y_i)$$


loss function

Typically, we consider functions of a parametric form:

$$\arg \min_{\theta} \frac{1}{n} \sum_{i=0}^n \ell(f(x_i; \theta), y_i)$$


model parameters

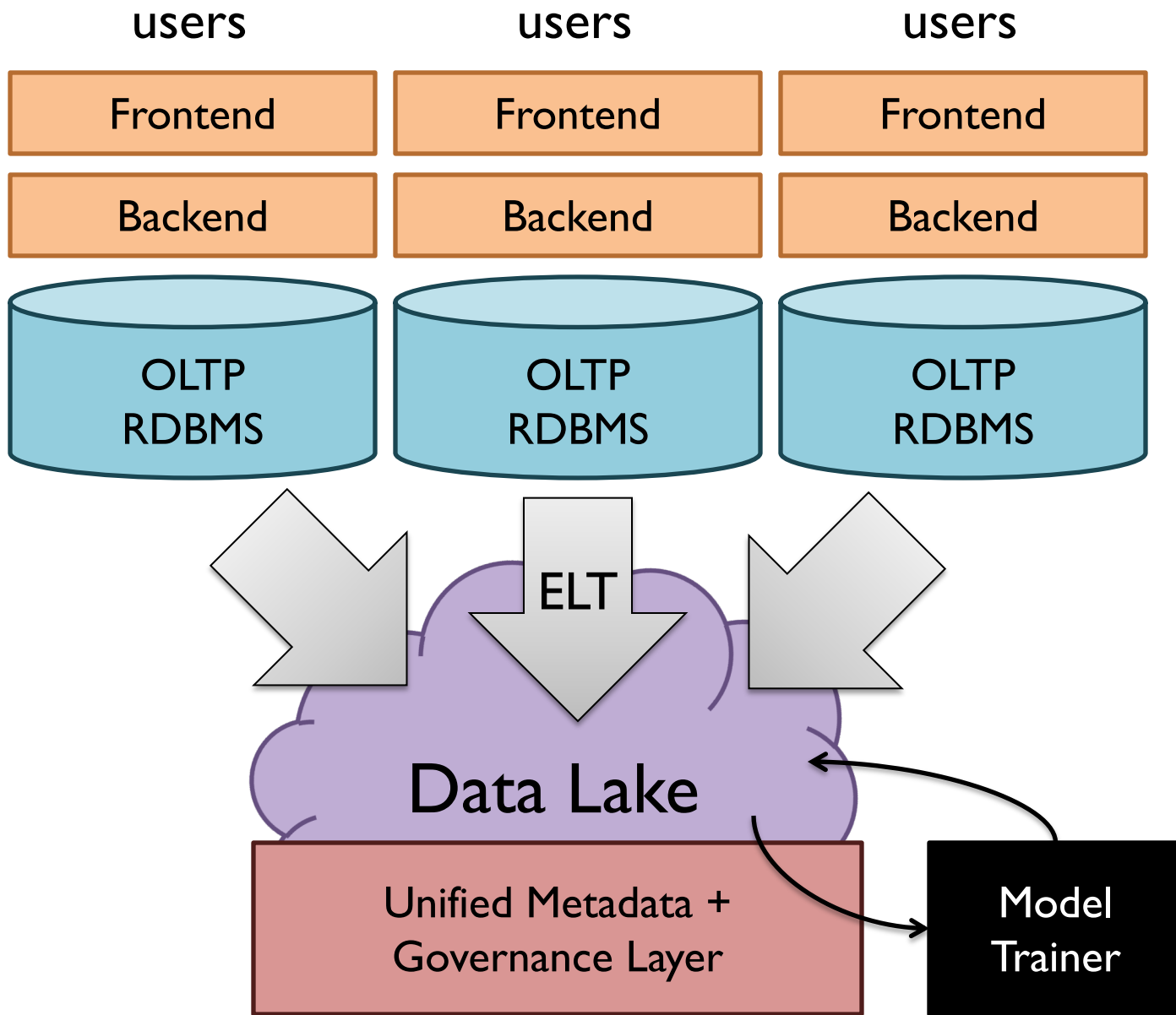
How do you do it?

Use `sklearn:model.fit(X, y)`

Didn't work? use `pkg1:model1.fit(X, y)`

Didn't work? use `pkg1:model2.fit(X, y)`

Didn't work? use `pkg2:modelA.fit(X, y)`



Lakehouse

Gradient Descent

(Batch) Gradient Descent

$$\theta^{(t+1)} \leftarrow \theta^{(t)} - \gamma^{(t)} \frac{1}{n} \sum_{i=0}^n \nabla \ell(f(\mathbf{x}_i; \theta^{(t)}), y_i)$$

“batch” learning: update model after considering all training instances

Stochastic Gradient Descent (SGD)

$$\theta^{(t+1)} \leftarrow \theta^{(t)} - \gamma^{(t)} \nabla \ell(f(\mathbf{x}; \theta^{(t)}), y)$$

“online” learning: update model after considering each (randomly selected) training instance

A scenic landscape of rolling green hills under a blue sky with large white clouds. The hills are covered in vibrant green grass, and the sky is filled with fluffy white clouds. The text "(Batch) Gradient Descent" is overlaid in the center of the image.

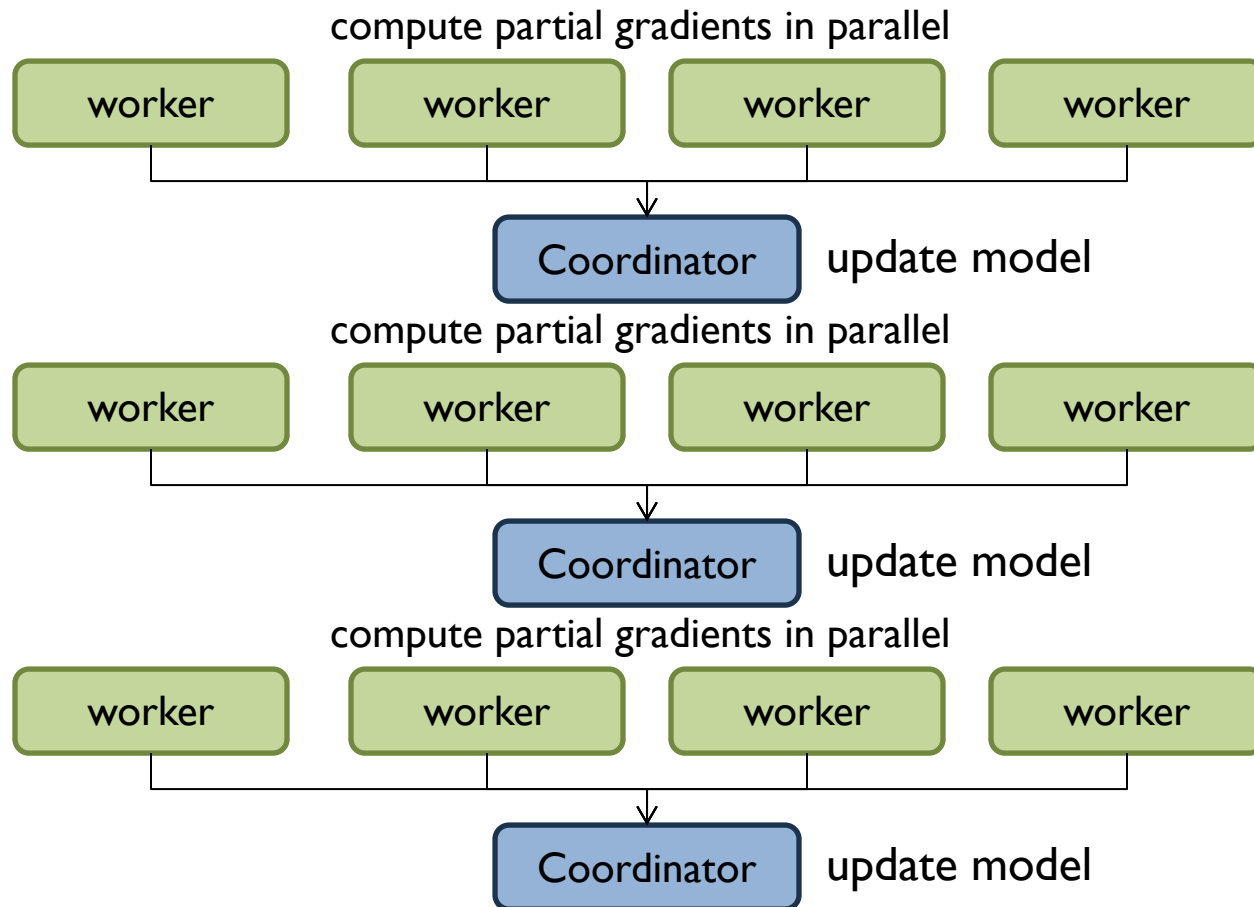
(Batch) Gradient Descent



Stochastic Gradient Descent

(Batch) Gradient Descent

$$\theta^{(t+1)} \leftarrow \theta^{(t)} - \gamma^{(t)} \frac{1}{n} \sum_{i=0}^n \nabla \ell(f(\mathbf{x}_i; \theta^{(t)}), y_i)$$



What's the
issue here?

Stochastic Gradient Descent

$$\theta^{(t+1)} \leftarrow \theta^{(t)} - \gamma^{(t)} \nabla \ell(f(\mathbf{x}; \theta^{(t)}), y)$$

single worker streams
through instances and
performs model updates



Important: Model update after every instance

How do you parallelize?

Model update after every instance



worker streams through instances and
perform model updates (uncoordinated)

Coordinator

update all models

Stochastic Gradient Descent w/ Mini-Batches

$$\theta^{(t+1)} \leftarrow \theta^{(t)} - \gamma^{(t)} \nabla \ell(f(\mathbf{x}; \theta^{(t)}), y)$$

single worker streams
through instances and
performs model updates



Important: Model update after every instance

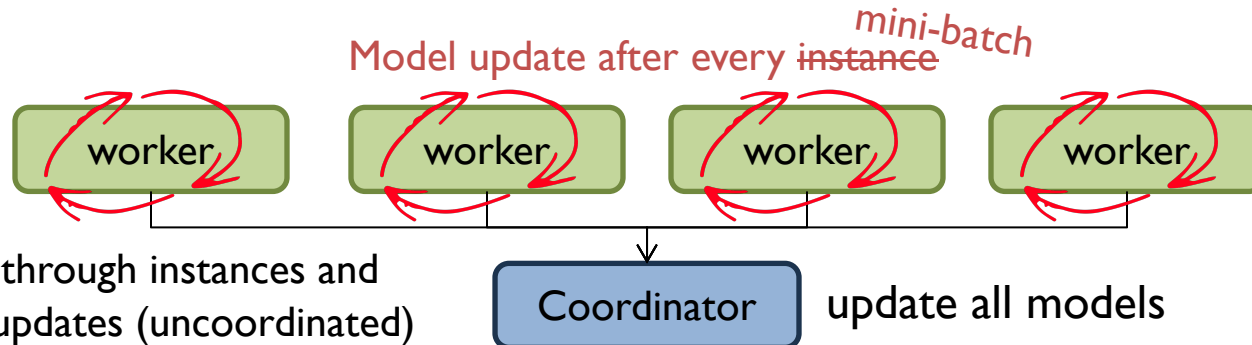
Problem: updates are very noisy

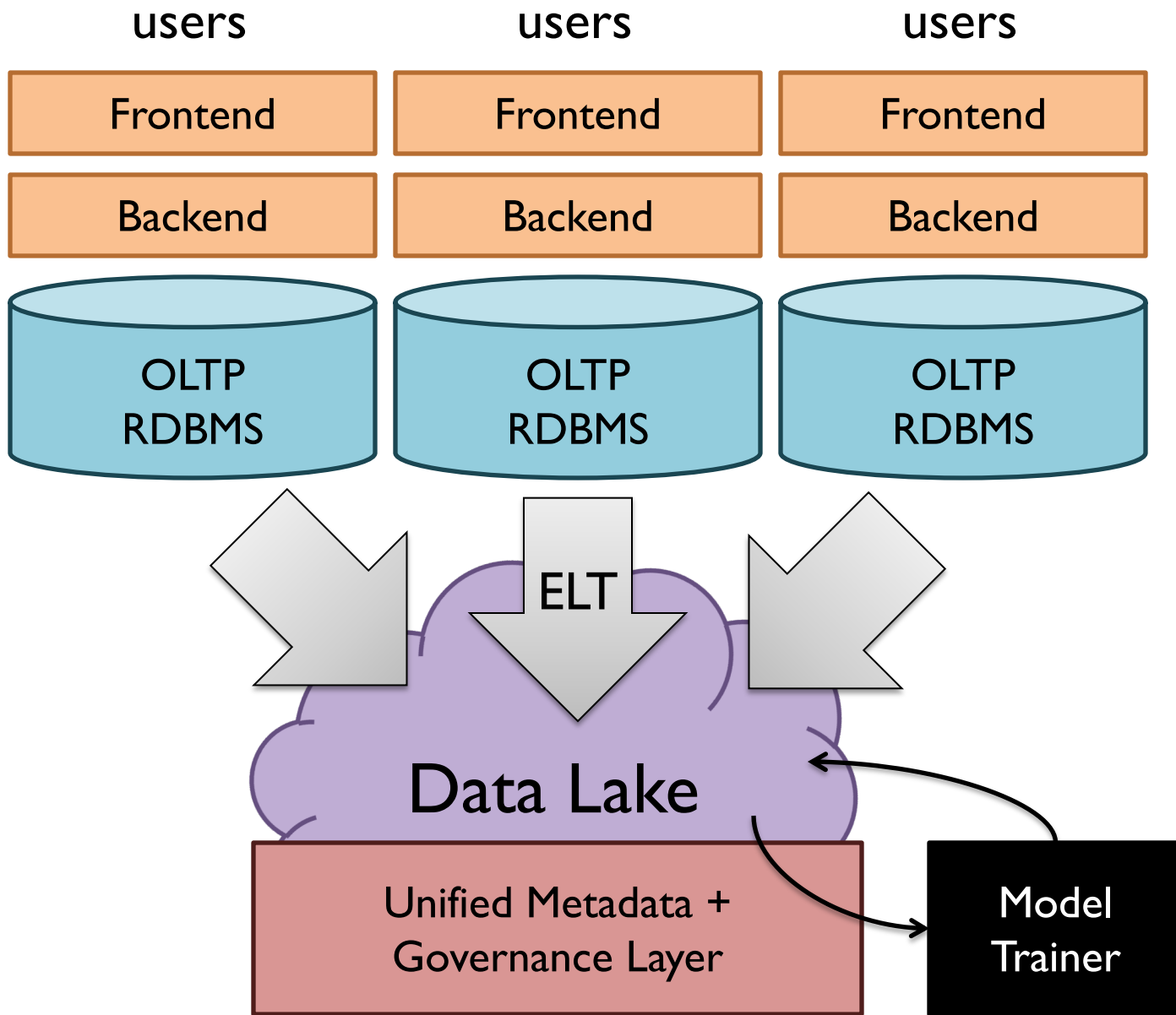
Solution: mini-batches

Divide dataset into small batches (e.g., 64)

Perform gradient descent on each mini-batch

Update model after processing each mini-batch





Lakehouse

Gather training data

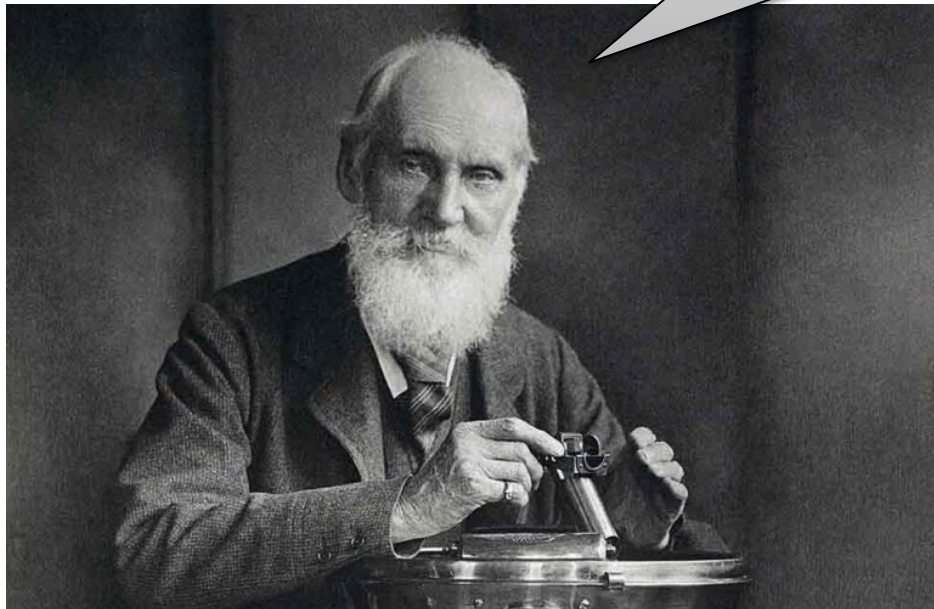
Train model

Deploy model

How do you know if it's better?
(Better than what?)

How do you know it's better?

If you can't measure it,
you can't improve it...



Define “better”

“Performance”

Faster: latency, throughput, etc.

Cheaper: cost per query, watt per query, etc.

More scalable: server load, memory usage, etc.

Higher-quality output

Define “higher quality”

Things you can measure but don’t have ground truth

Clickthrough rates

Time on site

...

Things you can measure but may have “ground truth”

Accuracy

Precision, recall

nDCG

...

Things that are difficult to measure

Quality of a summary

Quality of an answer to a question

Quality of an LLM response

...

tl;dr – evaluation is *really* hard
What are you trying to accomplish?

Avoid *post hoc* justifications
Tension between user and business goals

Okay, ^{assume} we know what to measure
Okay, ^{assume} we can measure it

Benchmark datasets

Static, can be internal or external

Batch evaluations

“Prospective”, internal data – but on the data platform

A/B tests

Prospective, internal data – but “in the wild”

When the data and the anecdotes disagree,
the anecdotes are usually right.

– Jeff Bezos

The Task

Given: $D = \{(\mathbf{x}_i, y_i)\}_i^n$



label

feature vector

$$\mathbf{x}_i = [x_1, x_2, x_3, \dots, x_d]$$

$$y \in \{0, 1\}$$

Why isn't this enough?

Induce: $f : X \rightarrow Y$

Such that loss is minimized

$$\frac{1}{n} \sum_{i=0}^n \ell(f(\mathbf{x}_i), y_i)$$


loss function

Typically, we consider functions of a parametric form:

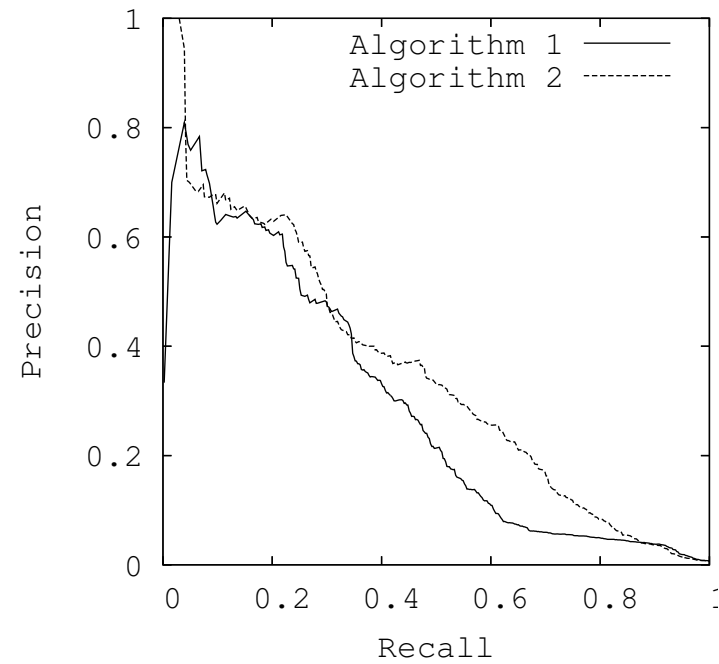
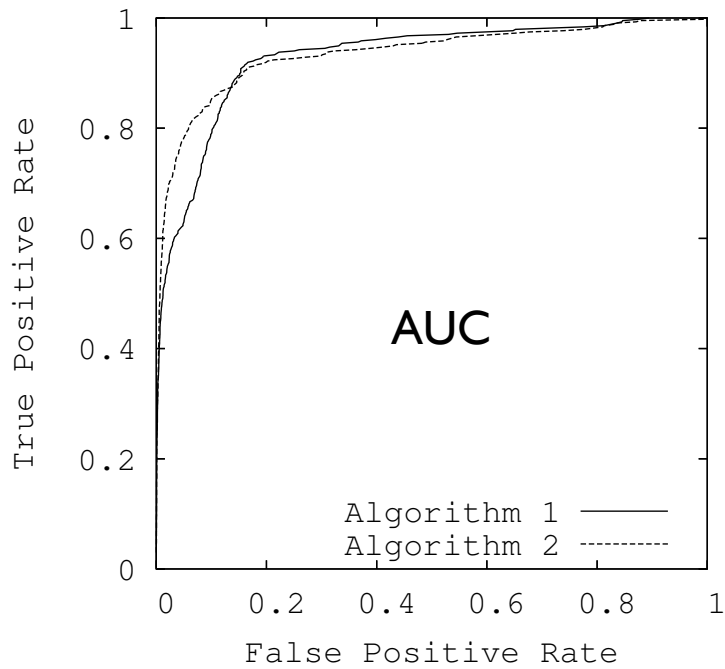
$$\arg \min_{\theta} \frac{1}{n} \sum_{i=0}^n \ell(f(x_i; \theta), y_i)$$


model parameters

Metrics

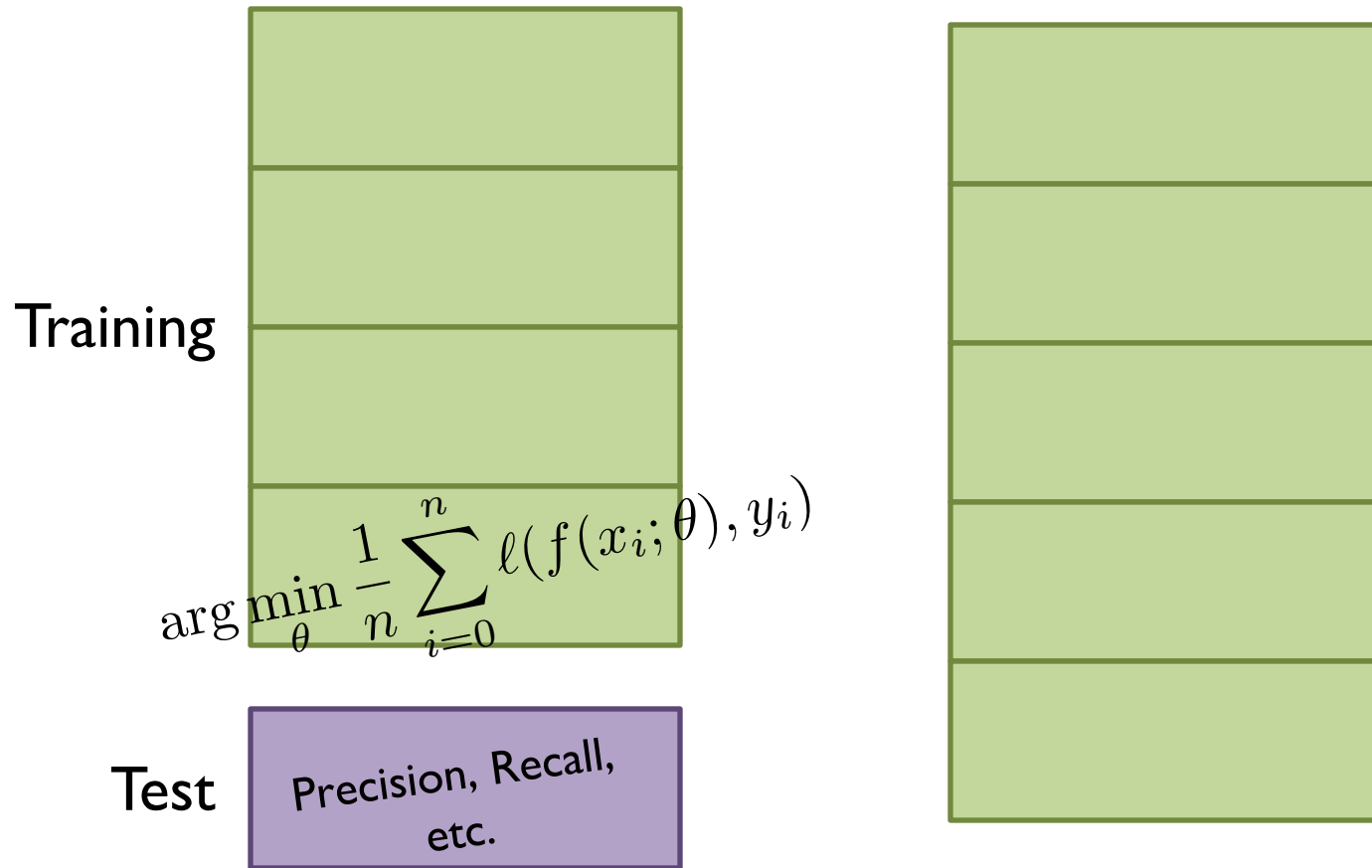
		Actual		
		Positive	Negative	
Predicted	Positive	True Positive (TP)	False Positive (FP) = Type I Error	Precision = $TP / (TP + FP)$
	Negative	False Negative (FN) = Type II Error	True Negative (TN)	Miss rate = $FN / (FN + TN)$
		Recall or TPR = $TP / (TP + FN)$	Fall-Out or FPR = $FP / (FP + TN)$	

ROC and PR Curves



Training/Testing Splits

Often, benchmark datasets



What happens if you need more?

Cross-Validation

“Prospective” Evaluations

Often, internal batch evaluations

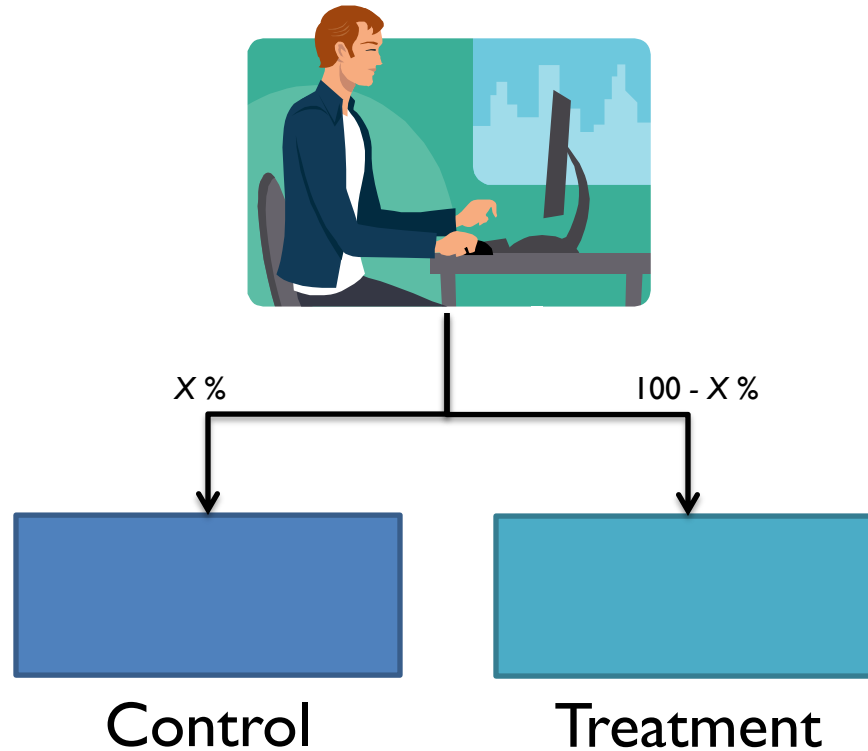


Why is this better?

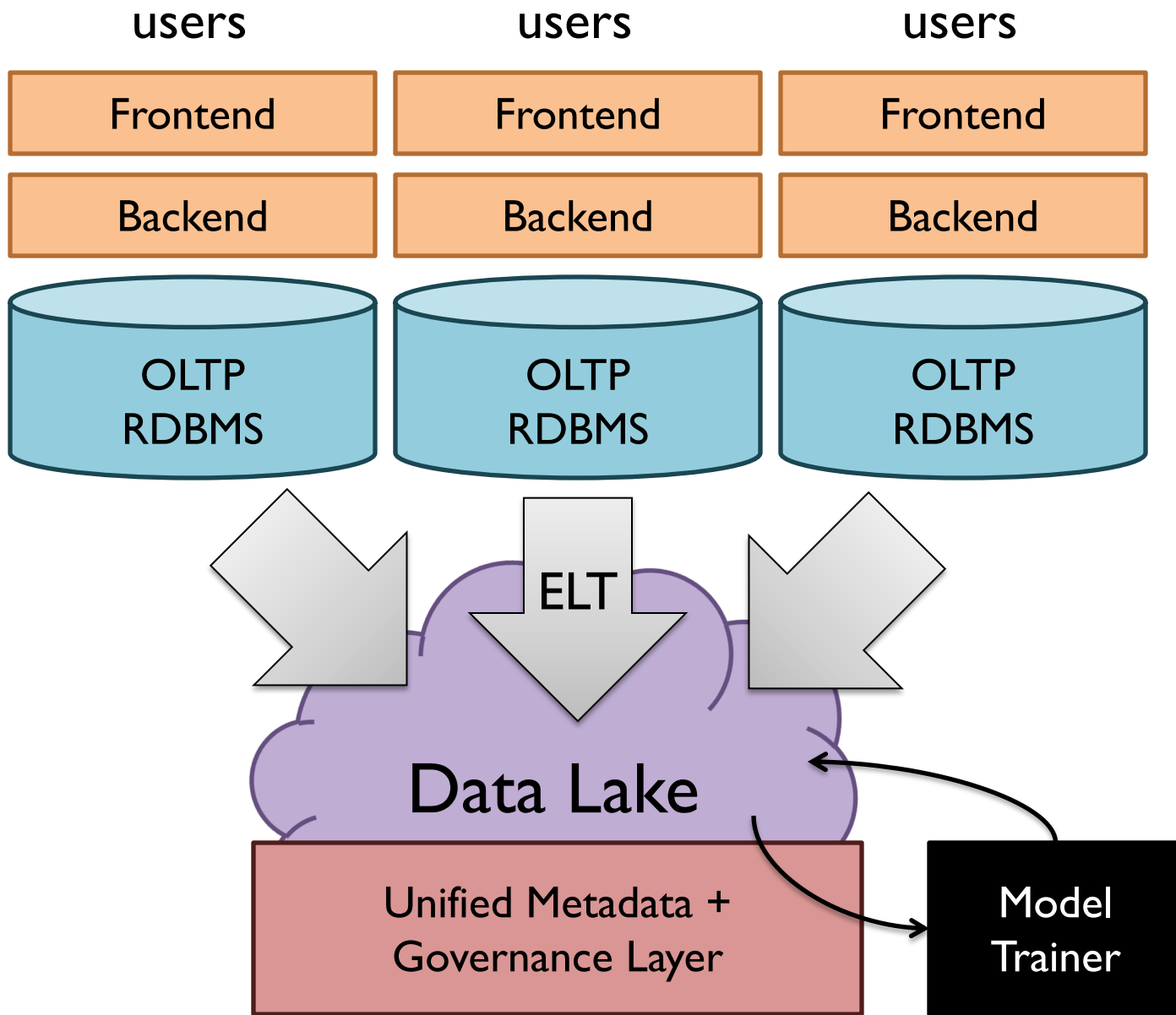
Where does this happen?

(Not ready for this yet...)

A/B Testing



Gather metrics, compare alternatives



Lakehouse

Gather training data

Train model

Deploy model

How do you know if it's better?

But if it's not?

Chance GO DIRECTLY
TO JAIL



DO NOT PASS GO, DO NOT COLLECT \$200

© 1936 PARKER BROTHERS

Components of a ML Solution

Data

Features

Model

Optimization

Gather training data

Train model

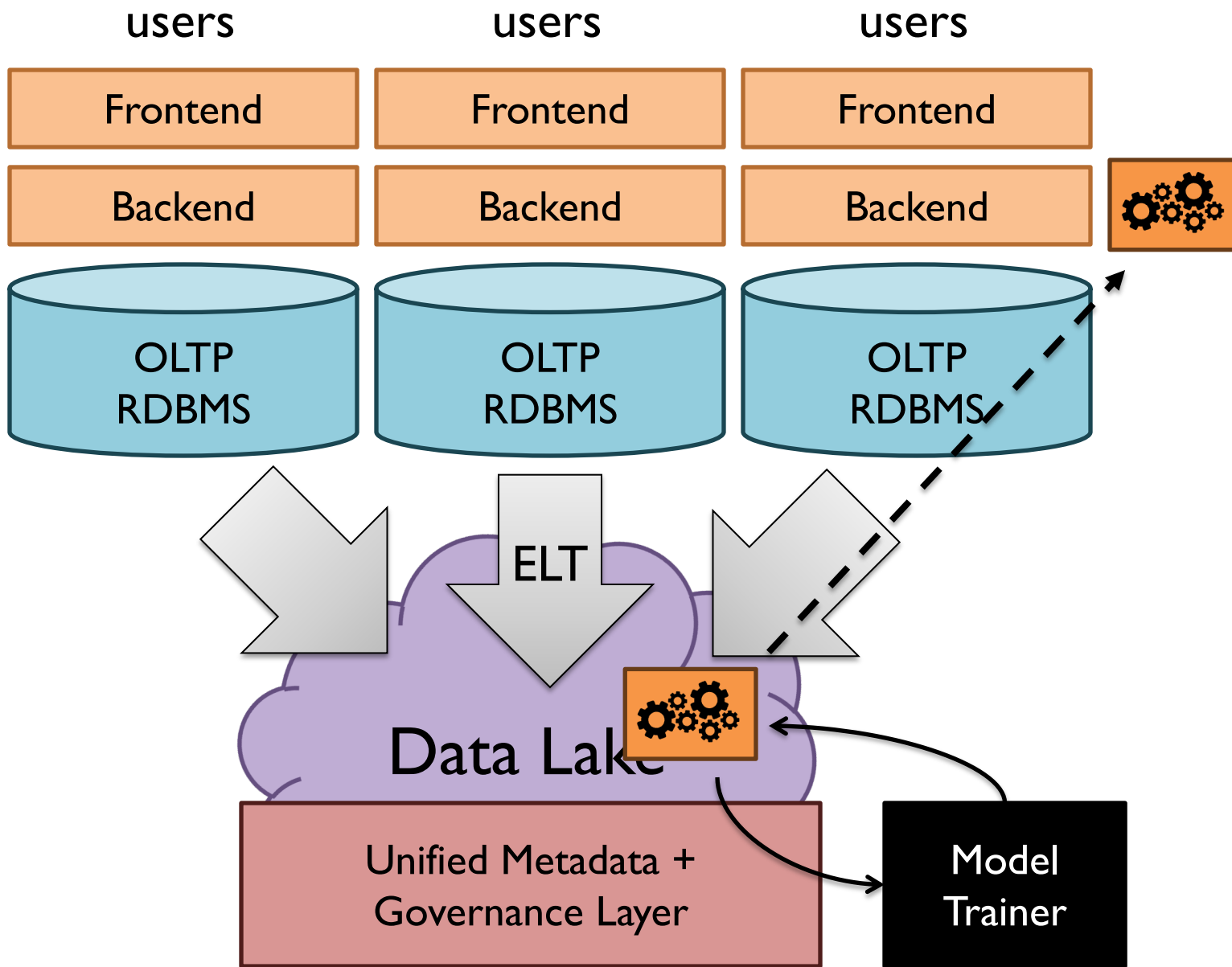
Deploy model

Okay, I really think it's better!

Gather training data

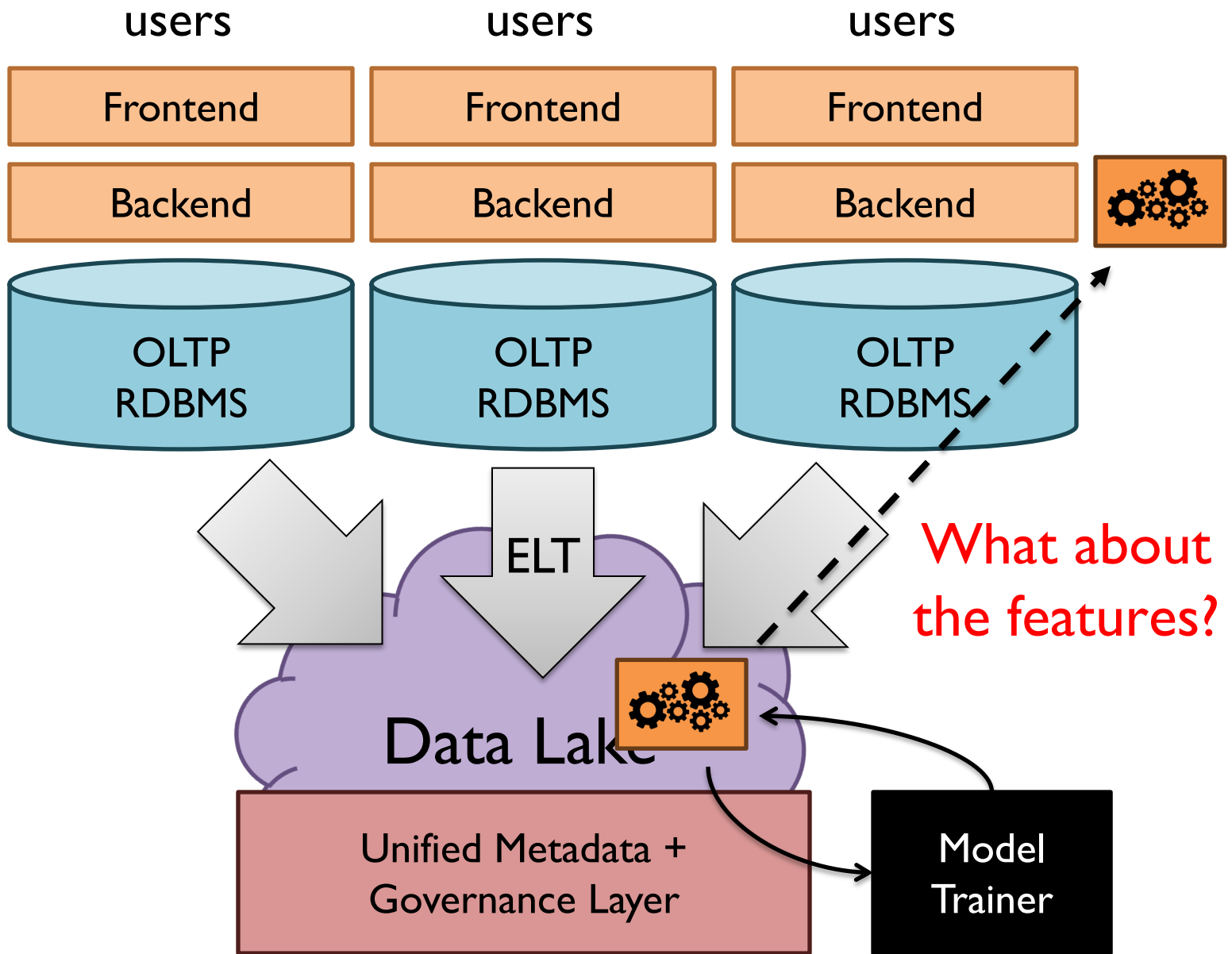
Train model

Deploy model



Lakehouse

How do you do it?
`model.deploy()`

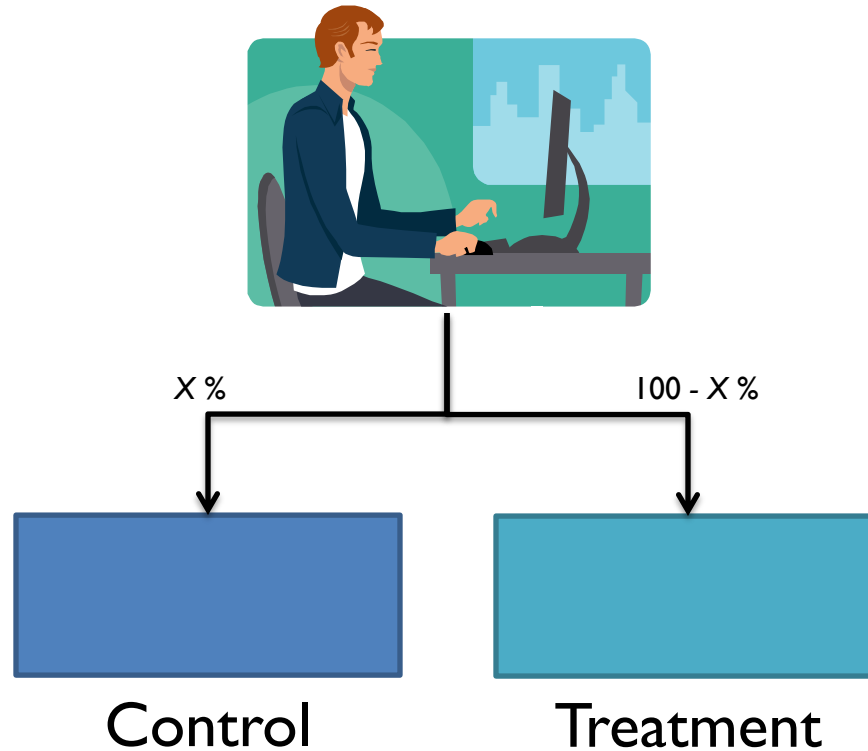


Lakehouse



Source: Wikipedia (Plumbing)

A/B Testing



Gather metrics, compare alternatives

Gather training data

Train model

Deploy model

Wait, you're not done yet!
Deployment is not a one-time thing...

How frequently is the model retrained?

What are its upstream dependencies?

What if training fails?

How frequently is the model deployed?

What if the deployment fails?

After deployment, what if the service stops working?

What if the quality degrades over time?

Gather training data
Train model
Deploy model

Goal for today: Dispel the myth of what a
data scientist *actually* does.
Applied {ML, AL} Researcher, etc.



Source: Wikipedia (Hills)



Source: Wikipedia (Plumbing)